💻 ARTIFICIAL INTELLIGENCE DEVELOPER INTERNSHIP PROGRAM

Hasnain Karimain Educational Academy

Software House & Training Center

Registered with SECP & PSEB (Ministry of IT, Government of Pakistan)

💻 AI Intern – Task 1

🎯 Task Title: Build a Multi-Class Sentiment Analysis System (Positive, Neutral, Negative, Mixed)

📝 Objective:

Develop an advanced sentiment analysis system that can classify text into **four categories**: Positive, Negative, Neutral, and Mixed. This task will test your skills in **Natural Language Processing (NLP), machine learning, and model evaluation** using real-world text data.

✅ Task Details:

You will build a **multi-class text classification model** that analyzes user input or dataset text and predicts sentiment as:

- Positive 😊
- Negative 😡
- Neutral 😐

- Mixed 🤔 (text containing both positive and negative opinions)

🛠️ Requirements:

🔹 Tools & Libraries:

- Python
- pandas
- scikit-learn
- nltk / TextBlob / spaCy (any one or combination)
- matplotlib / seaborn (for visualization)

🔄 Steps to Follow:

1️⃣ Data Collection:

- Create or collect a dataset with **at least 50–100 labeled sentences**
- Each sentence must be labeled as:
  - Positive
  - Negative
  - Neutral
  - Mixed

👉 Example:

Text	Sentiment
"I love the design but hate the performance"	Mixed
"This is amazing!"	Positive
"It's okay, nothing special"	Neutral
"Worst experience ever"	Negative

2️⃣ Data Preprocessing:

- Convert text to lowercase
- Remove punctuation
- Remove stopwords
- Tokenization
- Lemmatization (recommended)

3️⃣ Feature Engineering:

- Convert text into numerical form using:
    - CountVectorizer OR
    - TfidfVectorizer

4️⃣ Model Building:

- Train at least **one model**:
    - Logistic Regression ✅ (recommended)
    - OR Naive Bayes
    - OR SVM

5️⃣ Model Evaluation:

- Split dataset into training & testing (e.g., 80/20)
- Evaluate using:
    - Accuracy
    - Precision
    - Recall
    - F1-score

6️⃣ Prediction System:

- Create a **user input system**:

input("Enter your sentence: ")

- Output should be:

Predicted Sentiment: Mixed

📊 Expected Output Example:

Input:

"I like the features but the app is very slow"

Output:

Predicted Sentiment: Mixed

🔥 Bonus Features (Highly Recommended):

- ✅ Show **confusion matrix**
- ✅ Save trained model using `joblib` or `pickle`
- ✅ Display prediction probabilities
- ✅ Add batch prediction (multiple sentences)
- ✅ Create simple CLI menu system
- ✅ Export results to `.csv` file

📦 Submission Requirements:

Submit **ANY ONE** of the following:

- ✅ GitHub Repository (preferred)
- ✅ Google Colab Notebook
- ✅ ZIP Folder containing:
  - `.py` or `.ipynb` file
  - dataset (CSV)
  - README file

📁 Folder Structure (Recommended):

Sentiment_Classifier/
├── dataset.csv
├── model.py
├── utils.py
├── README.md
├── model.pkl (optional)

⏰ Deadline:

2–4 Hours

💻 AI Intern – Task 2

🎯 Task Title: Create a Text Preprocessing Pipeline with Custom Tokenization & Lemmatization

📝 Objective:

Build a **modular and reusable text preprocessing pipeline** that prepares raw text data for NLP tasks. This includes implementing **custom tokenization, stopword removal, and lemmatization**, similar to real-world AI systems.

✅ Task Details:

You will develop a **complete preprocessing system** that takes raw text input and outputs clean, structured text ready for machine learning models.

Your pipeline should handle:

- Cleaning raw text
- Custom tokenization (not just default library use)
- Stopword removal
- Lemmatization
- Output formatted processed text

🛠️ Requirements:

🔹 Tools & Libraries:

- Python
- nltk / spaCy (recommended)
- re (for regex cleaning)
- string

🔄 Steps to Follow:

1️⃣ Input Handling:

- Accept:
    - Single sentence input (via `input()`)

    - OR a list of sentences from a dataset (CSV/text file)

2️⃣ Text Cleaning:

- Convert to lowercase
- Remove:
    - Punctuation
    - Special characters
    - Numbers (optional)
- Remove extra spaces

3️⃣ Custom Tokenization:

- Implement your **own tokenizer** using:
    - Regex (`re.split`, `re.findall`) OR
    - Manual logic

👉 Example:

```
tokens = re.findall(r'\b\w+\b', text)
```

4️⃣ Stopword Removal:

- Use NLTK stopwords OR create your own custom list
- Remove common words like: *is, the, and, a, in*

5️⃣ Lemmatization:

- Use:
    - `WordNetLemmatizer` (NLTK) OR
    - spaCy lemmatizer

👉 Convert words to base form:

- running → run
- better → good

6️⃣ Pipeline Integration:

- Combine all steps into a **single reusable function or class**

👉 Example:

```
def preprocess(text):
  # cleaning
  # tokenization
  # stopword removal
  # lemmatization
  return processed_text
```

📊 Example:

🔹 Input:

"I am loving the new AI tools! They are running amazingly well."

🔹 Output:

['love', 'new', 'ai', 'tool', 'run', 'amazingly', 'well']

🔧 Technical Requirements:

- Code must be:
  - Clean
  - Well-commented
  - Modular (functions or class-based)
- Handle edge cases:
  - Empty input
  - Special characters
  - Mixed text

🔥 Bonus Features (Highly Recommended):

- ✅ Compare:
  - Raw text vs processed text
- ✅ Add stemming (PorterStemmer) and compare with lemmatization
- ✅ Process full dataset and save output to CSV
- ✅ Add logging system (show each step output)
- ✅ Create CLI menu:
  - 1: Single text
  - 2: Batch processing

📦 Submission Requirements:

Submit **ANY ONE**:

- ✅ GitHub Repository (preferred)
- ✅ Google Colab Notebook
- ✅ ZIP Folder containing:
    - `.py` / `.ipynb` file
    - sample dataset (optional)
    - README.md

📁 Recommended Folder Structure:

```
Text_Preprocessing_Pipeline/
├── main.py
├── preprocessing.py
├── dataset.csv (optional)
└── README.md
```

⏰ Deadline:

2–4 Hours

💻 AI Intern – Task 3

🎯 Task Title: Develop a Language Detection Model for Multilingual Text

📝 Objective:

Build a **machine learning-based language detection system** that can identify the language of a given text input. This task will test your skills in **text preprocessing, feature extraction, and multi-class classification** using multilingual data.

✅ Task Details:

You will create a model that can classify text into multiple languages such as:

- English
- Urdu
- Spanish
- French
- German
- (Add more if possible)

🛠️ Requirements:

🔹 Tools & Libraries:

- Python
- pandas
- scikit-learn
- nltk (optional)
- langdetect (optional for comparison only)

🔄 Steps to Follow:

1️⃣ Dataset Collection:

- Create or use a dataset with **at least 100–200 sentences**
- Each sentence should be labeled with its language

👉 Example:

Text	Language
"How are you?"	English
"آپ کیسے ہیں؟"	Urdu
"¿Cómo estás?"	Spanish
"Comment ça va?"	French

2️⃣ Data Preprocessing:

- Convert text to lowercase
- Remove unnecessary symbols (optional)
- Handle Unicode characters properly
- (Important) Do NOT remove important language-specific characters

3️⃣ Feature Engineering:

- Use:
  - **Character-level features (recommended)**
  - OR word-level features

👉 Use:

- CountVectorizer (with char n-grams)
- OR TfidfVectorizer

Example:

TfidfVectorizer(analyzer='char', ngram_range=(2,4))

4️⃣ Model Building:

Train at least one classification model:

- Naive Bayes ✅ (recommended)
- Logistic Regression
- SVM

5️⃣ Model Evaluation:

- Split dataset (80/20)
- Evaluate using:
    - Accuracy
    - Confusion Matrix
    - Classification Report

6️⃣ Prediction System:

- Create a user input system:

input("Enter text: ")

- Output:

Predicted Language: Urdu

📊 Example Output:

Input:

"Bonjour, comment allez-vous?"

Output:

Predicted Language: French

🔧 Technical Requirements:

- Code must be:
    - Clean
    - Well-structured
    - Properly commented
- Must handle:
    - Short text inputs
    - Mixed language edge cases (basic handling)

🔥 Bonus Features (Highly Recommended):

- ✅ Detect **top 2 probable languages** with probability scores
- ✅ Compare your model with `langdetect` library
- ✅ Save trained model using `joblib` or `pickle`
- ✅ Create batch prediction system (CSV input/output)
- ✅ Build simple CLI menu system

- ✅ Visualize confusion matrix using matplotlib/seaborn

📦 Submission Requirements:

Submit **ANY ONE**:

- ✅ GitHub Repository (preferred)
- ✅ Google Colab Notebook
- ✅ ZIP Folder containing:
  - `.py` / `.ipynb` file
  - dataset (CSV)
  - README.md

📁 Recommended Folder Structure:

```
Language_Detection_Model/
├── dataset.csv
├── train.py
├── predict.py
├── model.pkl (optional)
└── README.md
```

⏰ Deadline:

2–4 Hours

💻 AI Intern – Task 4

🎯 Task Title: Build a Fake News Detection System using Machine Learning

📝 Objective:

Develop a **machine learning-based fake news detection system** that can classify news articles or headlines as **Fake** or **Real**. This task focuses on **text classification, feature engineering, and real-world NLP problem solving**.

✅ Task Overview:

You will build a model that analyzes news content and predicts whether it is:

- Fake 🛑
- Real ✅

🛠️ Requirements:

🔹 Tools & Libraries:

- Python
- pandas
- scikit-learn
- nltk / re (for preprocessing)
- matplotlib / seaborn (for visualization)

🔄 Implementation Steps:

1️⃣ Dataset Collection:

Choose one:

- Use a dataset from Kaggle (Fake vs Real News)
- OR create a dataset with **at least 100–200 samples**

👉 Example:

Text	Label
"Government confirms new policy reform..."	Real
"Aliens landed in New York yesterday!"	Fake

2️⃣ Data Preprocessing:

- Convert text to lowercase
- Remove punctuation and special characters
- Remove stopwords
- Tokenization
- Lemmatization (recommended)

3️⃣ Feature Engineering:

Convert text into numerical features:

- **TfidfVectorizer** ✅ (recommended)
- OR CountVectorizer

4️⃣ Model Training:

Train at least one model:

- Logistic Regression ✅
- Naive Bayes
- Random Forest (bonus)

5️⃣ Model Evaluation:

- Split dataset (80/20)
- Evaluate using:
  - Accuracy
  - Precision
  - Recall
  - F1-score
  - Confusion Matrix

6️⃣ Prediction System:

Create a user input system:

```
news = input("Enter news text: ")
```

Output:

Prediction: Fake News

📊 Example:

🔹 Input:

"Breaking: Scientists confirm water found on Mars"

🔹 Output:

Prediction: Real News

🔧 Technical Requirements:

- Clean and well-commented code
- Proper modular structure
- Handle:
  - Short headlines
  - Long articles
  - Noisy text

🔥 Bonus Features (Highly Recommended):

- ✅ Show prediction probability (confidence score)
- ✅ Save trained model using `joblib` or `pickle`
- ✅ Create batch prediction system (CSV input/output)
- ✅ Visualize most important words (feature importance)
- ✅ Build simple CLI menu system
- ✅ Compare multiple models

📦 Submission Requirements:

Submit **ANY ONE**:

- ✅ GitHub Repository (preferred)
- ✅ Google Colab Notebook
- ✅ ZIP Folder containing:
  - `.py` / `.ipynb` file
  - dataset (CSV)
  - README.md

📁 Recommended Folder Structure:

Fake_News_Detector/
├── dataset.csv
├── train_model.py
├── predict.py
├── model.pkl
└── README.md

⏰ Deadline:

3–5 Hours

💻 AI Intern – Task 5

🎯 Task Title: Create an Email Classification System (Spam, Promotion, Social, Important)

📝 Objective:

Build a **multi-class email classification system** that can automatically categorize emails into different types such as **Spam, Promotion, Social, and Important**. This task simulates real-world email filtering systems like Gmail.

✅ Task Overview:

Your system should classify emails into:

- Spam 🚫
- Promotion 📢
- Social 👥
- Important ⭐

🛠️ Requirements:

🔹 Tools & Libraries:

- Python
- pandas
- scikit-learn
- nltk / re (for preprocessing)
- matplotlib / seaborn (optional for visualization)

🔄 Implementation Steps:

1️⃣ Dataset Preparation:

- Create or collect a dataset with **at least 150–300 email samples**
- Each email must have:
  - Email text/content
  - Label (Spam, Promotion, Social, Important)

👉 Example:

Email Text	Category
"Win a free iPhone now!!!"	Spam
"Flat 50% OFF on all products!"	Promotion
"John commented on your photo"	Social
"Meeting scheduled at 10 AM tomorrow"	Important

2️⃣ Data Preprocessing:

- Convert text to lowercase
- Remove punctuation & special characters
- Remove stopwords
- Tokenization

- Lemmatization (recommended)

3️⃣ Feature Engineering:

Convert text into numerical form using:

- **TfidfVectorizer** ✅ (recommended)
- OR CountVectorizer

4️⃣ Model Training:

Train at least one model:

- Logistic Regression ✅
- Naive Bayes
- Support Vector Machine (SVM)

5️⃣ Model Evaluation:

- Split dataset into training/testing (80/20)
- Evaluate using:
  - Accuracy
  - Precision
  - Recall
  - F1-score
  - Confusion Matrix

6️⃣ Prediction System:

Create a user input system:

email = input("Enter email text: ")

Output:

Category: Promotion

📊 Example:

🔹 Input:

"Congratulations! You have won a $500 gift card. Click here now!"

🔹 Output:

Category: Spam

🔧 Technical Requirements:

- Clean, well-commented code
- Modular structure (separate functions/files)
- Must handle:
    - Short messages
    - Long email content
    - Noisy text

🔥 Bonus Features (Highly Recommended):

- ✅ Show prediction probability (confidence score)
- ✅ Save model using `joblib` or `pickle`
- ✅ Batch classification (CSV file input/output)
- ✅ Display confusion matrix using seaborn
- ✅ Identify top keywords per category
- ✅ Create CLI menu (Single input / Bulk input)

📦 Submission Requirements:

Submit **ANY ONE**:

- ✅ GitHub Repository (preferred)
- ✅ Google Colab Notebook
- ✅ ZIP Folder containing:
    - `.py` / `.ipynb` file
    - dataset (CSV)
    - README.md

📁 Recommended Folder Structure:

```
Email_Classifier/
├── dataset.csv
├── train_model.py
├── predict.py
├── model.pkl
├── README.md
```

⏰ Deadline:

3–5 Hours

💻 AI Intern – Task 6

🎯 Task Title: Develop an Automatic Text Summarizer (Extractive + Abstractive Comparison)

📝 Objective:

Build a system that can automatically generate summaries of long text using **two different approaches**:

- **Extractive Summarization** (selecting important sentences)
- **Abstractive Summarization** (generating new summarized text like humans)

This task will help you understand **advanced NLP techniques and compare traditional vs modern AI approaches**.

✅ Task Overview:

You will create a summarization tool that:

- Takes long text as input
- Generates:
  - Extractive summary
  - Abstractive summary
- Compares both outputs

🛠️ Requirements:

🔹 Tools & Libraries:

- Python
- nltk / spaCy
- scikit-learn (for extractive methods)
- transformers (Hugging Face) ✅ (for abstractive summarization)
- pandas (optional)

🔄 Implementation Steps:

1️⃣ Input Data:

- Use:
    - Articles
    - Blog posts
    - News content

👉 At least **10–20 long text samples**

2️⃣ Text Preprocessing:

- Convert to lowercase
- Remove unnecessary symbols
- Sentence tokenization
- Word tokenization

🔹 PART 1: Extractive Summarization

3️⃣ Approach:

- Use techniques like:
    - TF-IDF scoring
    - Sentence ranking
    - Frequency-based scoring

👉 Select top important sentences as summary

4️⃣ Output Example:

Input:

"Artificial Intelligence is transforming industries by automating tasks and improving efficiency..."

Extractive Output:

"Artificial Intelligence is transforming industries. It improves efficiency and automates tasks."

🔹 PART 2: Abstractive Summarization

5️⃣ Approach:

- Use pretrained transformer models:

Examples:

- `t5-small`

- `bart-large-cnn`

👉 Use Hugging Face transformers pipeline

6️⃣ Output Example:

Abstractive Output:

"AI is revolutionizing industries by making processes more efficient."

🔄 Comparison Task:

7️⃣ Compare Both Methods:

- Accuracy (manual evaluation)
- Readability
- Length
- Meaning preservation

👉 Display results clearly

🔧 Technical Requirements:

- Code must be:
  - Clean
  - Modular
  - Well-commented
- Must include:
  - Both extractive & abstractive implementations
  - Clear comparison output

📊 Expected Output:

Original Text:
[Long paragraph]

Extractive Summary:
[Top sentences]

Abstractive Summary:
[Generated summary]

Comparison:

Extractive: More factual
Abstractive: More human-like

🔥 Bonus Features (Highly Recommended):

- ✅ Allow user input for custom text
- ✅ Show summary length control (short/medium/long)
- ✅ Save summaries to file (.txt or .csv)
- ✅ Build simple CLI menu
- ✅ Highlight selected sentences (extractive)
- ✅ Add ROUGE score evaluation (advanced)

📦 Submission Requirements:

Submit **ANY ONE**:

- ✅ GitHub Repository (preferred)
- ✅ Google Colab Notebook
- ✅ ZIP Folder containing:
  - `.py` / `.ipynb` file
  - sample texts
  - README.md

📁 Recommended Folder Structure:

```
Text_Summarizer/
├── extractive.py
├── abstractive.py
├── main.py
├── sample_texts.txt
└── README.md
```

⏰ Deadline:

4–6 Hours

💻 AI Intern – Task 7

🎯 Task Title: Train and Compare Multiple Models (Naive Bayes, SVM, Logistic Regression) for NLP Tasks

📝 Objective:

Build and evaluate multiple machine learning models for an NLP classification task, and **compare their performance** to understand which algorithm works best for text data.

This task focuses on **model training, evaluation, and comparative analysis**, which is a key skill in real-world AI development.

✅ Task Overview:

You will:

- Choose an NLP task (e.g., sentiment analysis, spam detection, or text classification)
- Train **three different models**:
    - Naive Bayes
    - Support Vector Machine (SVM)
    - Logistic Regression
- Compare their performance using evaluation metrics

🛠️ Requirements:

🔹 Tools & Libraries:

- Python
- pandas
- scikit-learn
- nltk / re (for preprocessing)
- matplotlib / seaborn (for visualization)

🔄 Implementation Steps:

1️⃣ Dataset Selection:

Choose one:

- Sentiment dataset (Positive/Negative)
- Spam detection dataset
- Fake news dataset
- OR create your own dataset (minimum 100–200 samples)

2️⃣ Data Preprocessing:

- Convert text to lowercase
- Remove punctuation
- Remove stopwords
- Tokenization
- Lemmatization (recommended)

3️⃣ Feature Engineering:

Convert text into numerical features:

- **TfidfVectorizer** ✅ (recommended)
- OR CountVectorizer

4️⃣ Model Training:

Train the following models:

- Multinomial Naive Bayes
- Support Vector Machine (SVM)
- Logistic Regression

5️⃣ Model Evaluation:

Evaluate all models using:

- Accuracy
- Precision
- Recall
- F1-score

👉 Store results in a table for comparison

6️⃣ Comparison Analysis:

Create a comparison table like:

Model	Accuracy	Precision	Recall	F1-score
Naive Bayes	85%	83%	84%	83%
SVM	88%	87%	86%	86%
Logistic Regression	90%	89%	88%	88%

👉 Identify the **best performing model**

7️⃣ Prediction System:

Create user input:

text = input("Enter text: ")

Output:

Best Model Prediction: Positive

📊 Example:

🔹 Input:

"This product is amazing and works perfectly!"

🔹 Output:

Naive Bayes: Positive
SVM: Positive
Logistic Regression: Positive

Best Model: Logistic Regression

🔧 Technical Requirements:

- Code must be:
  - Clean
  - Well-commented
  - Modular
- Must include:
  - All three models
  - Proper evaluation metrics
  - Comparison logic

🔥 Bonus Features (Highly Recommended):

- ✅ Plot comparison graph (bar chart of accuracy)
- ✅ Display confusion matrix for each model
- ✅ Perform cross-validation
- ✅ Save best model using `joblib`
- ✅ Add GridSearchCV for tuning
- ✅ Build CLI menu system

📦 Submission Requirements:

Submit **ANY ONE**:

- ✅ GitHub Repository (preferred)
- ✅ Google Colab Notebook
- ✅ ZIP Folder containing:
  - `.py` / `.ipynb` file
  - dataset (CSV)
  - README.md

📁 Recommended Folder Structure:

```
Model_Comparison_NLP/
├── dataset.csv
├── train_models.py
├── evaluate.py
├── best_model.pkl
└── README.md
```

⏰ Deadline:

3–5 Hours

💻 AI Intern – Task 8

🎯 Task Title: Implement Hyperparameter Tuning using GridSearchCV / RandomizedSearchCV

📝 Objective:

Improve the performance of a machine learning model by applying **hyperparameter tuning techniques**. This task focuses on optimizing models using **GridSearchCV and RandomizedSearchCV**, which are widely used in real-world AI systems.

✅ Task Overview:

You will:

- Select an NLP model (e.g., Logistic Regression, SVM, Naive Bayes)
- Define hyperparameters to tune
- Apply:
    - GridSearchCV (exhaustive search)
    - RandomizedSearchCV (efficient search)
- Compare model performance **before and after tuning**

🛠️ Requirements:

🔹 Tools & Libraries:

- Python
- pandas
- scikit-learn
- nltk / re (for preprocessing)
- matplotlib / seaborn (optional for visualization)

🔄 Implementation Steps:

1️⃣ Dataset Selection:

Use any NLP dataset:

- Sentiment analysis

- Spam detection
- Fake news detection
- OR your previous internship task dataset

👉 Minimum: **100–200 samples**

2️⃣ Data Preprocessing:

- Lowercase text
- Remove punctuation
- Remove stopwords
- Tokenization
- Lemmatization

3️⃣ Feature Engineering:

Convert text to numerical form using:

- **TfidfVectorizer ✅**

4️⃣ Baseline Model:

Train a basic model (without tuning):

- Logistic Regression OR
- SVM OR
- Naive Bayes

👉 Record baseline accuracy

5️⃣ Hyperparameter Tuning:

🔹 Option 1: GridSearchCV

- Define parameter grid

Example:

```
param_grid = {
  'C': [0.1, 1, 10],
  'solver': ['liblinear', 'lbfgs']
}
```

🔹 Option 2: RandomizedSearchCV

- Define parameter distribution

Example:

```
param_dist = {
  'C': [0.01, 0.1, 1, 10, 100],
  'penalty': ['l1', 'l2']
}
```

6️⃣ Model Training with Tuning:

- Apply GridSearchCV / RandomizedSearchCV
- Use cross-validation (cv=3 or 5)
- Find:
  - Best parameters
  - Best score

7️⃣ Performance Comparison:

Create a comparison like:

Model	Accuracy
Before Tuning	82%
After Tuning	89%

8️⃣ Prediction System:

text = input("Enter text: ")

Output:

Prediction (Tuned Model): Positive

📊 Example Output:

Best Parameters: {'C': 10, 'solver': 'liblinear'}
Best Accuracy: 89%

Before Tuning Accuracy: 82%
After Tuning Accuracy: 89%

🔧 Technical Requirements:

- Code must be:
  - Clean
  - Well-commented
  - Modular
- Must include:
  - Baseline model
  - Tuned model
  - Comparison

🔥 Bonus Features (Highly Recommended):

- ✅ Compare GridSearch vs RandomSearch performance
- ✅ Plot results (accuracy comparison graph)
- ✅ Use multiple models for tuning
- ✅ Save best model using `joblib`
- ✅ Show training time comparison
- ✅ Use Pipeline (Vectorizer + Model together)

📦 Submission Requirements:

Submit **ANY ONE**:

- ✅ GitHub Repository (preferred)
- ✅ Google Colab Notebook
- ✅ ZIP Folder containing:
  - `.py` / `.ipynb` file
  - dataset (CSV)
  - README.md

📁 Recommended Folder Structure:

```
Hyperparameter_Tuning/
├── dataset.csv
├── baseline_model.py
├── tuning.py
├── best_model.pkl
├── README.md
```

⏰ Deadline:

3–5 Hours

💻 AI Intern – Task 9

🎯 Task Title: Build a Model Evaluation Dashboard (Accuracy, Precision, Recall, F1)

📝 Objective:

Create an **interactive dashboard** that visualizes and compares the performance of machine learning models using key evaluation metrics such as **Accuracy, Precision, Recall, and F1-score**.This task focuses on **model evaluation, data visualization, and user interaction**, which are essential for real-world AI projects.

✅ Task Overview:

You will:

- Train or use existing ML models (from previous tasks)
- Calculate evaluation metrics
- Build a dashboard to display:
    - Model performance
    - Metric comparisons
    - Visual insights

🛠️ Requirements:

🔹 Tools & Libraries:

- Python
- pandas
- scikit-learn
- matplotlib / seaborn OR plotly
- **Streamlit (preferred)** ✅

🔄 Implementation Steps:

1️⃣ Dataset & Model:

- Use any dataset from previous tasks:
  - Sentiment analysis
  - Spam detection
  - Fake news detection
- Train at least **2–3 models**:
  - Logistic Regression
  - SVM
  - Naive Bayes

2️⃣ Evaluation Metrics:

Calculate:

- Accuracy
- Precision
- Recall
- F1-score

👉 Use:

from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

3️⃣ Data Preparation:

Create a structured dataset like:

Model	Accuracy	Precision	Recall	F1-score
Logistic Regression	90%	89%	88%	88%
SVM	88%	87%	86%	86%

4️⃣ Dashboard Development (Streamlit):

Build a simple UI that includes:

🔹 Sections:

- 📊 Model Comparison Table
- 📈 Bar Chart (metrics comparison)
- 🔍 Model selection dropdown
- 🧠 Display detailed metrics for selected model

5️⃣ Visualization:

- Bar chart comparing:
  - Accuracy
  - Precision
  - Recall
  - F1-score

👉 Use:

- matplotlib
- OR seaborn
- OR plotly

6️⃣ Sample Output (Dashboard):

Model Evaluation Dashboard

Select Model: Logistic Regression

Accuracy: 90%
Precision: 89%
Recall: 88%
F1-score: 88%

[Bar Chart Showing Comparison]

🔧 Technical Requirements:

- Clean, modular, well-commented code
- Functional Streamlit app (`app.py`)
- Proper metric calculation
- Clear UI layout

🔥 Bonus Features (Highly Recommended):

- ✅ Confusion matrix visualization
- ✅ Upload your own dataset feature
- ✅ Compare multiple models side-by-side
- ✅ Add ROC Curve (advanced)
- ✅ Export results as CSV
- ✅ Dark mode UI (Streamlit theme)

📦 Submission Requirements:

Submit **ANY ONE**:

- ✅ GitHub Repository (preferred)
- ✅ Streamlit Cloud live link ⭐
- ✅ Google Colab Notebook
- ✅ ZIP Folder containing:
    - `app.py`
    - dataset
    - model files
    - README.md

📁 Recommended Folder Structure:

```
Model_Evaluation_Dashboard/
├── app.py
├── dataset.csv
├── models/
├── utils.py
├── README.md
```

⏰ Deadline:

4–6 Hours

💻 AI Intern – Task 10

🎯 Task Title: Develop a Hybrid Recommendation System (Content-Based + Collaborative Filtering)

📝 Objective:

Build a **hybrid recommendation system** that combines:

- **Content-Based Filtering** (based on item features)
- **Collaborative Filtering** (based on user behavior)

This task simulates real-world systems used by platforms like Netflix, Amazon, and Spotify.

✅ Task Overview:

You will:

- Create or use a dataset of users, items (e.g., movies/products), and ratings
- Implement:
  - Content-based recommendation
  - Collaborative filtering
- Combine both methods into a **hybrid system**
- Generate personalized recommendations

🛠️ Requirements:

🔹 Tools & Libraries:

- Python
- pandas
- numpy
- scikit-learn
- matplotlib / seaborn (optional)

🔄 Implementation Steps:

1️⃣ Dataset Preparation:

Choose one:

- MovieLens dataset (recommended)
- OR create your own dataset

👉 Dataset should include:

- User ID
- Item (Movie/Product)
- Ratings
- (Optional: Genre, description)

2️⃣ Content-Based Filtering:

- Use item features (e.g., genre, description)
- Convert text/features using:
  - **TfidfVectorizer**
- Calculate similarity:
  - Cosine similarity

👉 Recommend items similar to what user likes

3️⃣ Collaborative Filtering:

Choose one approach:

- User-based filtering
- Item-based filtering

👉 Use:

- Cosine similarity
- OR sklearn NearestNeighbors

4️⃣ Hybrid System:

- Combine both methods:
  - Weighted average OR
  - Merge results

👉 Example:

Final Score = (0.5 * Content Score) + (0.5 * Collaborative Score)

5️⃣ Recommendation Output:

Generate at least **3–5 recommendations**

📊 Example Output:

Recommendations for User 1:

1. Inception
2. Interstellar
3. The Dark Knight
4. Tenet

🔧 Technical Requirements:

- Clean and modular code
- Separate logic for:
  - Content-based
  - Collaborative
  - Hybrid system
- Must include:
  - Similarity calculation
  - Recommendation function

🔥 Bonus Features (Highly Recommended):

- ✅ Add CLI input:
  - User selects movie → gets recommendations
- ✅ Visualize similarity matrix
- ✅ Add genre-based filtering
- ✅ Evaluate recommendation quality (basic)
- ✅ Save model/data using pickle
- ✅ Build simple UI using Streamlit

📦 Submission Requirements:

Submit **ANY ONE**:

- ✅ GitHub Repository (preferred)
- ✅ Google Colab Notebook
- ✅ ZIP Folder containing:
  - `.py` / `.ipynb` file
  - dataset
  - README.md

📁 Recommended Folder Structure:

Recommendation_System/
├── dataset.csv
├── content_based.py
├── collaborative.py
├── hybrid.py
├── main.py
├── README.md

⏰ Deadline:

4–6 Hours

💻 AI Intern – Task 11

🎯 Task Title: Build a Personalized Content Feed Engine (User Behavior Based)

📝 Objective:

Develop a **personalized content recommendation engine** that suggests content (e.g., posts, articles, videos) based on **user behavior and interaction history**.

This task simulates real-world systems used by platforms like social media feeds and news apps.

✅ Task Overview:

You will:

- Track user behavior (likes, clicks, views, ratings)
- Analyze user preferences
- Recommend personalized content dynamically

🛠️ Requirements:

🔹 Tools & Libraries:

- Python
- pandas
- numpy
- scikit-learn
- matplotlib / seaborn (optional)

🔄 Implementation Steps:

1️⃣ Dataset Preparation:

Create or use a dataset with:

- User ID
- Content ID (Post/Video/Article)
- Interaction type:

    - Like 👍
    - View 👀
    - Click 🔗
    - Share 🔁
- (Optional: Content category like Tech, Sports, AI, Business)

👉 Example:

User	Content	Category	Interaction
U1	Post1	AI	Like
U1	Post2	Tech	View
U2	Post3	Sports	Click

2️⃣ Behavior Scoring System:

Assign weights to user actions:

```
weights = {
  "view": 1,
  "click": 2,
  "like": 3,
  "share": 4
}
```

👉 Convert interactions into numerical scores

3️⃣ User Profile Creation:

- Build a **user preference profile** based on:
    - Categories
    - Interaction scores

👉 Example:
User likes AI content → recommend more AI posts

4️⃣ Content Representation:

- Represent content using:
    - Categories
    - OR text features (TF-IDF)

5️⃣ Recommendation Logic:

Use:

- Cosine similarity
- OR simple ranking based on user preference scores

👉 Recommend top **5 relevant content items**

6️⃣ Personalized Feed Output:

Recommended Feed for User U1:

1. AI Trends 2025
2. Latest Tech Innovations
3. Machine Learning Guide
4. Startup Growth Tips

🔧 Technical Requirements:

- Clean, modular code
- Separate logic for:
  - Data processing
  - User profiling
  - Recommendation engine
- Must include:
  - Behavior scoring
  - Personalized output

📊 Example:

🔹 Input:

User ID: U1

🔹 Output:

Top Recommendations:
- AI News Update
- Deep Learning Basics
- Tech Industry Insights

🔥 Bonus Features (Highly Recommended):

- ✅ Real-time update of recommendations after new interaction
- ✅ Add time-based decay (recent activity more important)
- ✅ Visualize user preferences (bar chart)
- ✅ Build simple CLI interface
- ✅ Filter by category
- ✅ Store data in CSV/JSON

📦 Submission Requirements:

Submit **ANY ONE**:

- ✅ GitHub Repository (preferred)
- ✅ Google Colab Notebook
- ✅ ZIP Folder containing:
  - `.py` / `.ipynb` file
  - dataset
  - README.md

📁 Recommended Folder Structure:

```
Content_Feed_Engine/
├── dataset.csv
├── user_profile.py
├── recommender.py
├── main.py
└── README.md
```

⏰ Deadline: 4–6 Hours

🚀 AI / Backend Internship – Task 12

🎯 Task Title: Create a Product Recommendation API using FastAPI

📝 Objective:

Build a **REST API using FastAPI** that provides **product recommendations** based on user preferences using a simple recommendation logic (content-based or similarity-based).

This task simulates how real-world AI systems expose models as APIs (like Amazon, Netflix, etc.).

✅ Task Overview:

You will:

- Create a FastAPI backend
- Build a product dataset
- Implement a recommendation logic
- Expose endpoints for:
    - Product listing
    - Recommendations
    - User input-based suggestions

🛠️ Requirements:

🔹 Tools & Libraries:

- Python
- FastAPI
- uvicorn
- pandas
- scikit-learn

Install:

pip install fastapi uvicorn pandas scikit-learn

🔄 Implementation Steps:

1️⃣ Dataset Creation:

Create a simple product dataset:

Product ID	Name	Category	Description
1	iPhone 14	Mobile	Apple smartphone with camera
2	Samsung S23	Mobile	Android flagship phone
3	MacBook Pro	Laptop	Apple laptop for professionals

👉 Minimum: **10–20 products**

2️⃣ Feature Engineering:

Convert product text into vectors:

- Use **TF-IDF Vectorizer**
- Combine:
  - Name
  - Category
  - Description

3️⃣ Recommendation Logic:

Use:

- Cosine Similarity
- OR Nearest Neighbors

👉 Based on product similarity

4️⃣ FastAPI Setup:

Basic structure:

```
from fastapi import FastAPI

app = FastAPI()
```

5️⃣ API Endpoints:

🔹 1. Root Endpoint

```
@app.get("/")
def home():
  return {"message": "Product Recommendation API is running"}
```

🔹 2. Get All Products

```
@app.get("/products")
def get_products():
  return products_list
```

🔹 3. Recommend Products

```
@app.get("/recommend/{product_name}")
def recommend(product_name: str):
  # find similarity
  return top_recommendations
```

🔹 4. User-Based Recommendation (Optional)

```
@app.post("/recommend/user")
def recommend_for_user(user_preferences: dict):
  return recommended_products
```

📊 Example Output:

🔹 Request:

```
/recommend/iPhone 14
```

🔹 Response:

```
{
 "recommendations": [
   "iPhone 13",
   "Samsung S23",
   "Google Pixel 7"
 ]
}
```

🔧 Technical Requirements:

- Clean API structure

- Modular code (separate logic file recommended)
- Proper response formatting (JSON)
- Handle errors (invalid product names)

🔥 Bonus Features (Highly Recommended):

- ✅ Add search API (`/search?query=`)
- ✅ Save model using pickle
- ✅ Add user history tracking
- ✅ Dockerize the API
- ✅ Add Swagger UI documentation (auto in FastAPI)
- ✅ Deploy on Render / Railway / HuggingFace Spaces

📦 Submission Requirements:

Submit:

- ✅ GitHub Repository (preferred)
- OR
- ✅ ZIP Folder containing:
  - `main.py`
  - dataset file
  - requirements.txt
  - README.md

📁 Recommended Folder Structure:

```
Product_Recommendation_API/
├── main.py
├── recommender.py
├── dataset.csv
├── requirements.txt
└── README.md
```

⏰ Deadline: 4–6 Hours

🤖 AI Intern – Task 13

🎯 Task Title: Build a Context-Aware AI Chatbot using NLP Techniques

📝 Objective:

Develop a context-aware chatbot that can:

- **Understand user queries**
- **Maintain conversation context**
- **Provide intelligent, relevant responses**

👉 This simulates real-world chatbots used in customer support, assistants, and AI products.

✅ Task Overview:

You will:

- **Preprocess user input**
- **Use NLP techniques for understanding intent**
- **Maintain conversation context (memory)**
- **Generate meaningful responses**

🛠️ Tech Stack:

- **Python**
- **nltk / spaCy**
- **scikit-learn**
- **(Optional) transformers / pretrained models**
- **Flask / FastAPI (optional for deployment)**

🔄 Step-by-Step Implementation

1️⃣ Basic Dataset (Intents)

Create a simple intent dataset:

```
intents = {
```

```
    "greeting": ["hi", "hello", "hey"],
    "goodbye": ["bye", "see you"],
    "weather": ["weather today", "is it hot?"],
    "name": ["what is your name"]
}
```

2️⃣ Text Preprocessing

```
import nltk
from nltk.tokenize import word_tokenize
from nltk.stem import WordNetLemmatizer
lemmatizer = WordNetLemmatizer()
def preprocess(text):
    tokens = word_tokenize(text.lower())
    tokens = [lemmatizer.lemmatize(w) for w in tokens]
    return " ".join(tokens)
```

3️⃣ Feature Extraction (TF-IDF)

```
from sklearn.feature_extraction.text import TfidfVectorizer
sentences = []
labels = []
```

```
for intent, examples in intents.items():
    for ex in examples:
        sentences.append(preprocess(ex))
        labels.append(intent)


vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(sentences)
```

4️⃣ Train Intent Classifier

```
from sklearn.linear_model import LogisticRegression
model = LogisticRegression()
model.fit(X, labels)
```

5️⃣ Context Management (Important Part)

👉 Store previous conversation

```
context = {}
def update_context(user_id, intent):
    context[user_id] = intent
```

6️⃣ Response System

```
responses = {
    "greeting": "Hello! How can I help you?",
```

```
    "goodbye": "Goodbye! Have a nice day!",
    "weather": "Please tell me your city.",
    "name": "I am your AI assistant."}
```

7️⃣ Chat Function

```
def chatbot(user_input, user_id="user1"):
    processed = preprocess(user_input)
    vec = vectorizer.transform([processed])
    intent = model.predict(vec)[0]
    # Context-aware logic
    if context.get(user_id) == "weather":
        return f"Weather info for {user_input} coming soon..."
    update_context(user_id, intent)
    return responses.get(intent, "Sorry, I didn't understand.")
```

8️⃣ Run Chatbot

```
while True:
    user_input = input("You: ")
    if user_input.lower() == "exit":
        break
    print("Bot:", chatbot(user_input))
```

📊 Example Conversation:

You: hi

Bot: Hello! How can I help you?

You: weather

Bot: Please tell me your city.

You: Karachi

Bot: Weather info for Karachi coming soon...

🔧 Technical Requirements:

- **Intent classification working**
- **Context memory implemented**
- **Clean and modular code**
- **NLP preprocessing used**

🔥 Bonus Features (Highly Recommended):

🚀 Advanced Upgrades:

- **✅ Use spaCy instead of NLTK**
- **✅ Add Named Entity Recognition (NER)**
- **✅ Use pretrained models (BERT / transformers)**
- **✅ Build web UI (Streamlit or Flask)**
- **✅ Add voice input/output**
- **✅ Store conversation history in database**

📦 Submission:

Submit:

- **Python file / Notebook**
- **Dataset (intents)**
- **README**

📁 Folder Structure:

AI_Chatbot/

│── intents.json

│── model.py

│── chatbot.py

│── utils.py

│── README.md

⏰ Deadline: 5–7 Hours

🤖 AI Intern – Task 14

🎯 Task Title: Develop an Intent Classification System using Machine Learning

📝 Objective:

Build a system that can automatically detect user intent from text input using machine learning techniques.

👉 Example:

- **"Hi" → Greeting**
- **"Order pizza" → Food Order**
- **"What's the weather?" → Weather Query**

This is a core component of chatbots, virtual assistants, and NLP systems.

✅ Task Overview:

You will:

- **Create an intent dataset**
- **Preprocess text**
- **Convert text into numerical features**
- **Train a classification model**
- **Predict intent for new user input**

🛠️ Tech Stack:

- **Python**
- **pandas**
- **scikit-learn**
- **nltk / spaCy**

🔄 Step-by-Step Implementation

1️⃣ Create Dataset

import pandas as pd

```
data = {
  "text": [
      "hi", "hello", "hey",
      "bye", "goodbye",
      "order pizza", "I want food",
      "weather today", "is it raining",
      "what is your name"
  ],
  "intent": [
      "greeting","greeting","greeting",
      "goodbye","goodbye",
      "food_order","food_order",
      "weather","weather",
      "name"
  ]
}


df = pd.DataFrame(data)
```

2️⃣ Text Preprocessing

```
import re
```

```
def preprocess(text):
    text = text.lower()
    text = re.sub(r'[^a-zA-Z ]', '', text)
    return text

df["clean_text"] = df["text"].apply(preprocess)
```

3️⃣ Feature Extraction (TF-IDF)

```
from sklearn.feature_extraction.text import TfidfVectorizer

vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(df["clean_text"])
y = df["intent"]
```

4️⃣ Train Model

👉 Use any model:

- Logistic Regression ✅
- Naive Bayes
- SVM

```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression()
model.fit(X, y)
```

5️⃣ Prediction Function

```
def predict_intent(text):
    text = preprocess(text)
    vec = vectorizer.transform([text])
    return model.predict(vec)[0]
```

6️⃣ Test System

```
while True:
    user_input = input("You: ")
    if user_input.lower() == "exit":
        break
    print("Intent:", predict_intent(user_input))
```

📊 Example Output:

```
You: hello
Intent: greeting

You: I want pizza
Intent: food_order

You: is it hot today
Intent: weather
```

🔧 Technical Requirements:

- Proper preprocessing
- TF-IDF feature extraction
- Working classification model
- Clean, modular code

🔥 Bonus Features (Highly Recommended):

🚀 Advanced Improvements:

- ✅ Use multiple models & compare accuracy
- ✅ Add train-test split:

from sklearn.model_selection import train_test_split

- ✅ Evaluate model:

from sklearn.metrics import classification_report

- ✅ Use spaCy for better preprocessing
- ✅ Add confidence score (`predict_proba`)
- ✅ Save model using `joblib`
- ✅ Integrate with chatbot (Task 13)

📦 Submission:

Include:

- Python file / Notebook
- Dataset
- README

📁 Folder Structure:

Intent_Classifier/

│── dataset.csv

│── model.py

│── utils.py

├── main.py

└── README.md

⏰ Deadline: 3–5 Hours

🤖 AI Intern – Task 15

🎯 Task Title: Create a Domain-Specific Q&A Chatbot using TF-IDF + Cosine Similarity

📝 Objective:

Build a chatbot that answers user questions based on a specific domain dataset using:

- **TF-IDF Vectorization**
- **Cosine Similarity**

👉 This is a retrieval-based chatbot (like FAQ bots used in websites, universities, banks, etc.)

✅ Task Overview:

You will:

- **Create a domain-specific Q&A dataset**
- **Preprocess text**
- **Convert text into vectors**
- **Match user query with most relevant question**
- **Return the best answer**

🛠️ Tech Stack:

- **Python**
- **pandas**
- **scikit-learn**
- **nltk / re**

🔄 Step-by-Step Implementation

1️⃣ Create Dataset (Q&A)

```
import pandas as pd



data = {

  "question": [

    "What is AI?",

    "What is machine learning?",

    "What is Python?",

    "What is data science?",

    "How to learn AI?"

  ],

  "answer": [

    "AI is artificial intelligence.",

    "Machine learning is a subset of AI.",

    "Python is a programming language.",

    "Data science is data analysis field.",
```

```
        "Start with Python and ML basics."
    ]
}

df = pd.DataFrame(data)
```

2 Text Preprocessing

```
import re

def preprocess(text):
    text = text.lower()
    text = re.sub(r'[^a-zA-Z ]', '', text)
    return text
df["clean_question"] = df["question"].apply(preprocess)
```

3 TF-IDF Vectorization

```
from sklearn.feature_extraction.text import TfidfVectorizer

vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(df["clean_question"])
```

4 Cosine Similarity Function

```
from sklearn.metrics.pairwise import cosine_similarity
```

5️⃣ Chatbot Function

```
def chatbot(query):
    query = preprocess(query)
    query_vec = vectorizer.transform([query])

    similarity = cosine_similarity(query_vec, X)
    index = similarity.argmax()

    return df.iloc[index]["answer"]
```

6️⃣ Run Chatbot

```
while True:
    user_input = input("You: ")
    if user_input.lower() == "exit":
        break

    print("Bot:", chatbot(user_input))
```

📊 Example Conversation:

You: what is AI

Bot: AI is artificial intelligence.

You: explain python

Bot: Python is a programming language.

🔧 Technical Requirements:

- **TF-IDF implemented correctly**
- **Cosine similarity used for matching**
- **Clean preprocessing**
- **Functional chatbot loop**

⚠️ Important Limitation (Explain in README):

- **This chatbot:**
  - **Does NOT generate answers**
  - **Only retrieves best match from dataset**

🔥 Bonus Features (Highly Recommended):

🚀 Advanced Upgrades:

- **✅ Add similarity threshold:**

```
if similarity.max() < 0.3:

  return "Sorry, I don't understand."
```

- **✅ Use larger dataset (100+ Q&A pairs)**
- **✅ Add multiple domains (education, health, tech)**
- **✅ Use n-grams in TF-IDF**
- **✅ Build Streamlit UI**
- **✅ Highlight matched question**
- **✅ Store dataset in CSV**

📦 Submission:

Include:

- **Python file / Notebook**
- **Dataset (CSV preferred)**
- **README**

📁 Folder Structure:

QA_Chatbot/

│── qa_dataset.csv

│── chatbot.py

│── utils.py

│── main.py

│── README.md

⏰ Deadline: 3–5 Hours

🖼️ AI Intern – Task 16

🎯 Task Title: Build an Image Classification System using Transfer Learning (ResNet / MobileNet)

📝 Objective:

Build a **deep learning image classification system** using **transfer learning** with pretrained models like:

- ResNet50
- MobileNetV2

👉 You will classify images into at least **2–10 categories** using a real dataset or custom dataset.

✅ Task Overview:

You will:

- Load image dataset
- Preprocess images
- Use pretrained model (Transfer Learning)
- Train classification head
- Evaluate model performance
- Test predictions on new images

🛠️ Tech Stack:

- Python
- TensorFlow / Keras ⭐
- NumPy
- Matplotlib
- (Optional) OpenCV

🔄 Step-by-Step Implementation

1️⃣ Dataset Selection

Choose one:

- Cats vs Dogs dataset 🐱🐶
- Flowers dataset 🌸
- CIFAR-10 dataset
- OR custom dataset (recommended 2–5 classes minimum)

👉 Folder structure example:

dataset/

├── cats/

├── dogs/

├── birds/

2️⃣ Import Libraries

```
import tensorflow as tf

from tensorflow.keras.preprocessing.image import ImageDataGenerator

from tensorflow.keras.applications import MobileNetV2

from tensorflow.keras import layers, models
```

3️⃣ Image Preprocessing

```
datagen = ImageDataGenerator(

  rescale=1./255,

  validation_split=0.2

)
```

4️⃣ Load Dataset

```
train_data = datagen.flow_from_directory(
    "dataset/",
    target_size=(224,224),
    batch_size=32,
    class_mode='categorical',
    subset='training'
)

val_data = datagen.flow_from_directory(
    "dataset/",
    target_size=(224,224),
    batch_size=32,
    class_mode='categorical',
    subset='validation'
)
```

5️⃣ Load Pretrained Model (Transfer Learning)

👉 Using MobileNetV2:

```
base_model = MobileNetV2(
    weights='imagenet',
```

```
    include_top=False,
    input_shape=(224,224,3)
)
base_model.trainable = False
```

6️⃣ Build Custom Model

```
model = models.Sequential([
    base_model,
    layers.GlobalAveragePooling2D(),
    layers.Dense(128, activation='relu'),
    layers.Dense(train_data.num_classes, activation='softmax')
])
```

7️⃣ Compile Model

```
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
```

8️⃣ Train Model

```
history = model.fit(
```

```
    train_data,
    validation_data=val_data,
    epochs=5
)
```

9️⃣ Evaluate Model

```
loss, acc = model.evaluate(val_data)
print("Accuracy:", acc)
```

🔟 Make Predictions

```
import numpy as np
from tensorflow.keras.preprocessing import image

img = image.load_img("test.jpg", target_size=(224,224))
img_array = image.img_to_array(img)/255.0
img_array = np.expand_dims(img_array, axis=0)

prediction = model.predict(img_array)
print(prediction)
```

📊 Example Output:

```
Predicted Class: Cat
```

Accuracy: 92%

🔧 Technical Requirements:

- Proper use of transfer learning
- Image preprocessing (resizing, normalization)
- Train/validation split
- Functional prediction system

🔥 Bonus Features (Highly Recommended):

🚀 Advanced Upgrades:

- ✅ Fine-tune last layers of MobileNet/ResNet
- ✅ Add confusion matrix
- ✅ Plot accuracy/loss graphs
- ✅ Real-time webcam prediction (OpenCV)
- ✅ Save model:

model.save("image_model.h5")

- ✅ Streamlit UI for image upload
- ✅ Data augmentation (rotation, zoom, flip)

📦 Submission:

Include:

- Python / Notebook file
- Dataset (or link)
- README

📁 Folder Structure:

Image_Classifier/

│── dataset/

│── train.py

│── predict.py

├── model.h5

└── README.md

⏰ **Deadline: 4–6 Hours**

🎯 AI Intern – Task 17

🧠 Task Title: Develop an Object Detection Model using YOLO or SSD

📝 Objective:

Build a real-time object detection system using a pretrained deep learning model such as:

- **YOLO (You Only Look Once) ⭐ Recommended**
- **SSD (Single Shot Detector)**

👉 The model should detect and classify multiple objects in images or video frames.

✅ Task Overview:

You will:

- **Load a pretrained object detection model**
- **Run detection on images or video**
- **Draw bounding boxes around objects**
- **Display class labels and confidence scores**
- **Test on real-world images/video**

🛠️ Tech Stack:

- **Python**
- **OpenCV**
- **YOLOv5 / YOLOv8 OR SSD**
- **PyTorch (for YOLOv5/8) OR TensorFlow (SSD)**
- **Matplotlib (optional)**

🔄 Step-by-Step Implementation

1️⃣ Install Requirements

YOLOv5 (Recommended)

pip install torch torchvision opencv-python

pip install ultralytics

2️⃣ Load YOLO Model

```
from ultralytics import YOLO

model = YOLO("yolov8n.pt")  # pretrained model
```

3️⃣ Run Object Detection on Image

```
results = model("test.jpg")

results.show()
```

4️⃣ Run Detection on Video

```
import cv2

cap = cv2.VideoCapture("video.mp4")

while cap.isOpened():

  ret, frame = cap.read()

  if not ret:

    break

  results = model(frame)

  annotated_frame = results[0].plot()

  cv2.imshow("YOLO Detection", annotated_frame)

  if cv2.waitKey(1) & 0xFF == ord('q'):

    break

cap.release()

cv2.destroyAllWindows()
```

5️⃣ Extract Detection Data

```
for r in results:
```

```
for box in r.boxes:
    print(box.cls, box.conf)
```

📊 Example Output:

Detected Objects:

- Person (0.92)

- Car (0.88)

- Dog (0.85)

🧠 What YOLO Does:

- **Detects multiple objects in real time**
- **Draws bounding boxes**
- **Gives confidence score**
- **Very fast compared to traditional models**

🔧 Technical Requirements:

- **Use pretrained YOLO or SSD model**
- **Process image/video input**
- **Display bounding boxes correctly**
- **Show class + confidence score**

🔥 Bonus Features (Highly Recommended):

🚀 Advanced Upgrades:

- **✅ Use webcam real-time detection:**

```
model.predict(source=0, show=True)
```

- **✅ Save output video with detections**
- **✅ Filter specific classes (e.g., only "person")**
- **✅ Count objects in frame**
- **✅ Build Streamlit UI for upload detection**
- **✅ Fine-tune YOLO on custom dataset**
- **✅ Save detection logs in CSV**

📦 Submission:

Include:

- **Python script / Notebook**
- **Sample image/video**
- **README**

📁 Folder Structure:

Object_Detection_YOLO/

│── test.jpg

│── video.mp4

│── detect.py

│── requirements.txt

│── README.md

⏰ Deadline: 4–6 Hours

🎯 AI Intern – Task 18

🧠 Task Title: Create an Image Similarity Search Engine using Feature Extraction

📝 Objective:

Build an **Image Similarity Search System** that finds visually similar images based on a query image using **deep learning feature extraction**.

👉 Example: Upload a shoe image → system returns similar shoes.

✅ Task Overview:

You will:

- Extract features from images using pretrained CNN
- Store feature vectors
- Compare similarity using distance metrics
- Return top-N similar images

🛠️ Tech Stack:

- Python
- OpenCV / PIL
- NumPy
- Scikit-learn
- TensorFlow / PyTorch
- Pretrained Model (ResNet50 / MobileNet / VGG16)

🔄 Step-by-Step Implementation

1️⃣ Install Dependencies

```
pip install numpy opencv-python scikit-learn tensorflow
```

2️⃣ Load Pretrained Model (Feature Extractor)

```
from tensorflow.keras.applications.resnet50 import ResNet50, preprocess_input

from tensorflow.keras.models import Model

import numpy as np

import cv2

# Load model without classification layer

base_model = ResNet50(weights='imagenet', include_top=False, pooling='avg')
```

3️⃣ Image Preprocessing Function

```
def preprocess_image(img_path):

    img = cv2.imread(img_path)

    img = cv2.resize(img, (224, 224))

    img = preprocess_input(img)

    return np.expand_dims(img, axis=0)
```

4️⃣ Extract Features

```
def extract_features(img_path):

    img = preprocess_image(img_path)

    features = base_model.predict(img)

    return features.flatten()
```

5️⃣ Build Feature Database

```
import os

image_folder = "dataset/"

features_list = []
```

```
image_paths = []

for file in os.listdir(image_folder):

    path = os.path.join(image_folder, file)

    features = extract_features(path)

    features_list.append(features)

    image_paths.append(path)

features_list = np.array(features_list)
```

6️⃣ Similarity Search (Cosine Similarity)

```
from sklearn.metrics.pairwise import cosine_similarity

def find_similar(query_img, top_k=5):

    query_features = extract_features(query_img)

    similarities = cosine_similarity([query_features], features_list)[0]

    indices = np.argsort(similarities)[::-1][:top_k]

    results = [(image_paths[i], similarities[i]) for i in indices]

    return results
```

7️⃣ Test the System

```
results = find_similar("query.jpg")



for img, score in results:

    print(img, score)
```

📊 Example Output:

dataset/shoe1.jpg → 0.95

dataset/shoe2.jpg → 0.91

dataset/shoe3.jpg → 0.88

🧠 How It Works:

- CNN extracts **deep visual features**
- Converts image → vector (embeddings)
- Compares vectors using similarity metric
- Returns closest matches

🔧 Technical Requirements:

- Use pretrained CNN (ResNet / MobileNet)
- Feature extraction pipeline
- Similarity comparison (cosine / Euclidean)
- Return Top-K results

🔥 Bonus Features (Highly Recommended):

🚀 Advanced Upgrades:

- ✅ Use FAISS for fast similarity search
- ✅ Build Streamlit UI (upload image + results)
- ✅ Display images instead of paths
- ✅ Use CLIP model for semantic similarity
- ✅ Add category filtering
- ✅ Save features in `.pkl` file
- ✅ Use GPU acceleration

📦 Submission:

Include:

- Python script / Notebook
- Sample dataset (10–50 images)
- Query image
- README

📁 Folder Structure:

Image_Similarity_Engine/

├── dataset/

├── query.jpg

├── main.py

├── features.pkl

├── requirements.txt

└── README.md

⏰ Deadline:

4–6 Hours

🎯 AI Intern – Project 19

🧠 Project Title: Fine-Tune a Transformer Model (BERT/DistilBERT) for Sentiment Analysis

📝 Objective:

Fine-tune a pretrained transformer model such as **BERT** or **DistilBERT** to perform **sentiment analysis** on text data.

👉 The model should classify text into:

- Positive
- Negative
- Neutral *(optional: multi-class support)*

✅ Project Overview:

You will:

- Load a pretrained transformer model
- Prepare and preprocess a labeled dataset
- Fine-tune the model on sentiment classification
- Evaluate model performance
- Test predictions on custom input text

🛠️ Tech Stack:

- Python
- Hugging Face Transformers 🤗
- PyTorch / TensorFlow
- Scikit-learn
- Pandas / NumPy

🔄 Step-by-Step Implementation

1️⃣ Install Dependencies

pip install transformers datasets torch scikit-learn pandas

2️⃣ Load Dataset

```
from datasets import load_dataset

dataset = load_dataset("imdb")  # binary sentiment dataset
```

3️⃣ Load Tokenizer

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("distilbert-base-uncased")
```

4️⃣ Tokenization Function

```
def tokenize(example):
    return tokenizer(example['text'], padding='max_length', truncation=True)


dataset = dataset.map(tokenize, batched=True)
```

5️⃣ Load Pretrained Model

```
from transformers import AutoModelForSequenceClassification

model = AutoModelForSequenceClassification.from_pretrained(
    "distilbert-base-uncased",
    num_labels=2
)
```

6️⃣ Training Setup

```
from transformers import TrainingArguments

training_args = TrainingArguments(
    output_dir="./results",
```

```
    learning_rate=2e-5,
    per_device_train_batch_size=16,
    num_train_epochs=2,
    evaluation_strategy="epoch"
)
```

7️⃣ Train Model

```
from transformers import Trainer
trainer = Trainer(
    model=model,
    args=training_args,  train_dataset=dataset["train"].shuffle(seed=42).select(range(5000)),
eval_dataset=dataset["test"].shuffle(seed=42).select(range(1000)))trainer.train()
```

8️⃣ Evaluate Model

```
trainer.evaluate()
```

9️⃣ Test Prediction

```
text = "This product is amazing!"
inputs = tokenizer(text, return_tensors="pt", truncation=True, padding=True)
outputs = model(**inputs)
import torch
pred = torch.argmax(outputs.logits, dim=1).item()
print("Predicted Label:", pred)
```

📊 Example Output:

Input: "This movie was fantastic!"

Output: Positive ✅

🧠 How It Works:

- Transformer models understand **context and semantics**
- Tokenizer converts text into model-readable format
- Fine-tuning adapts pretrained knowledge to your dataset
- Output layer predicts sentiment class

🔧 Technical Requirements:

- Use pretrained BERT or DistilBERT
- Implement tokenization pipeline
- Fine-tune model on dataset
- Evaluate using standard metrics

🔥 Bonus Features (Highly Recommended):

🚀 Advanced Enhancements:

- ✅ Convert to multi-class sentiment (Positive/Neutral/Negative)
- ✅ Use custom dataset (Twitter, reviews, etc.)
- ✅ Add confusion matrix & classification report
- ✅ Deploy with Streamlit or FastAPI
- ✅ Save & reload trained model
- ✅ Use GPU (CUDA) for faster training

📦 Submission:

Include:

- Python Notebook / Script
- Dataset (or dataset link)
- Trained model files
- README with explanation

📁 Folder Structure:

BERT_Sentiment_Project/

├── data/

├── model/

├── train.py

├── app.py (optional)

├── requirements.txt

└── README.md

⏰ **Deadline: 5–7 Hours**

🎯 AI Intern – Project 20

🧠 Project Title: Build a Text Generation Model using GPT-style Architecture (Hugging Face)

📝 Objective:

Develop a **text generation system** using a **GPT-style pretrained transformer model** from Hugging Face.

👉 The model should generate coherent and contextually relevant text based on a given prompt.

✅ Project Overview:

You will:

- Load a pretrained GPT-style model
- Tokenize input text properly
- Generate meaningful text outputs
- Control generation parameters (length, temperature, etc.)
- Test the model with different prompts

🛠️ Tech Stack:

- Python
- Hugging Face Transformers 🤗
- PyTorch / TensorFlow
- Datasets (optional)
- Streamlit (optional for UI)

🔄 Step-by-Step Implementation

1️⃣ Install Dependencies

```
pip install transformers torch
```

2️⃣ Load GPT Model

```
from transformers import pipeline

generator = pipeline("text-generation", model="gpt2")
```

3️⃣ Generate Text

```
prompt = "Artificial Intelligence will change the future because"

output = generator(
    prompt,
    max_length=100,
    num_return_sequences=1)

print(output[0]["generated_text"])
```

4️⃣ Advanced Configuration

```
output = generator(
    prompt,
    max_length=150,
```

temperature=0.8,

top_k=50,

top_p=0.95,

do_sample=True)

📊 Example Output:

Artificial Intelligence will change the future because it enables machines to learn, adapt, and make decisions...

🧠 How It Works:

- GPT is a transformer-based language model
- It predicts the next word based on previous context
- Uses probabilistic sampling for text generation
- Produces human-like coherent text

🔧 Technical Requirements:

- Use Hugging Face GPT-style model (GPT-2 or similar)
- Implement text generation pipeline
- Control generation parameters
- Test multiple prompts

🔥 Bonus Features (Highly Recommended):

🚀 Advanced Enhancements:

- ✅ Fine-tune GPT on custom dataset
- ✅ Build Streamlit chatbot UI

- ✅ Add prompt templates system
- ✅ Compare GPT-2 vs GPT-Neo outputs
- ✅ Save generated outputs to file
- ✅ Deploy model using FastAPI

📦 Submission:

Include:

- Python script / Notebook
- Sample prompts & outputs
- README with explanation

📁 Folder Structure:

GPT_Text_Generation_Project/

│── model.py

│── app.py (optional)

│── prompts.txt

│── requirements.txt

│── README.md

⏰ Deadline:

5–7 Hours

🎯 AI Intern – Project 21

🧠 Project Title: Create a Named Entity Recognition (NER) System using Transformers

📝 Objective:

Develop a Named Entity Recognition (NER) system using Transformer-based models from Hugging Face.

👉 The system should identify and classify entities in text such as:

- **Person (PER)**
- **Organization (ORG)**
- **Location (LOC)**
- **Date / Time (optional)**

✅ Project Overview:

You will:

- **Load a pretrained Transformer NER model**
- **Tokenize input text properly**
- **Extract named entities from sentences**
- **Display entity labels with confidence scores**
- **Test on custom user inputs**

🛠️ Tech Stack:

- **Python**
- **Hugging Face Transformers 🤗**
- **PyTorch / TensorFlow**

- SpaCy (optional for comparison)
- Pandas

🔄 Step-by-Step Implementation

1️⃣ Install Dependencies

```
pip install transformers torch
```

2️⃣ Load Pretrained NER Model

```
from transformers import pipeline


ner_pipeline = pipeline("ner", grouped_entities=True)
```

3️⃣ Test on Sample Text

```
text = "Elon Musk founded Tesla in California in 2003."


result = ner_pipeline(text)


for entity in result:
  print(entity)
```

4️⃣ Output Format

Entity: Elon Musk → PERSON

Entity: Tesla → ORGANIZATION

Entity: California → LOCATION

5️⃣ Custom Input Function

```
def extract_entities(text):
    results = ner_pipeline(text)

    for entity in results:
        print(f"{entity['word']} → {entity['entity_group']}")
```

6️⃣ Run Interactive System

```
while True:
    user_input = input("Enter text: ")

    if user_input.lower() == "exit":
        break

    extract_entities(user_input)
```

🧠 How It Works:

- Transformer model understands context of words
- Identifies named entities using deep learning
- Groups tokens into meaningful entity labels
- Returns structured entity predictions

🔧 Technical Requirements:

- Use pretrained Transformer-based NER model
- Extract entities from text input
- Display entity type and value
- Handle multiple entities per sentence

🔥 Bonus Features (Highly Recommended):

🚀 Advanced Enhancements:

- ✅ Use `dbmdz/bert-large-cased-finetuned-conll03-english` model
- ✅ Highlight entities in colored text
- ✅ Build Streamlit NER dashboard
- ✅ Compare spaCy vs Transformer NER
- ✅ Save extracted entities to CSV file
- ✅ Add confidence score filtering

📦 Submission:

Include:

- Python script / Notebook
- Sample input/output
- README file

📁 Folder Structure:

NER_System_Project/

│── ner.py

├── app.py (optional)

├── requirements.txt

├── README.md

⏰ Deadline:

4–6 Hours

🎯 AI Intern – Task 22

🧠 Task Title: Build and Deploy an AI Model using Streamlit Dashboard

📝 Objective:

Develop and deploy an AI/ML model using a Streamlit dashboard to create an interactive web application where users can input data and get real-time predictions.

👉 The goal is to turn a trained ML model into a fully working web app.

✅ Task Overview:

You will:

- **Train or load a machine learning model**
- **Build a Streamlit web interface**
- **Accept user input through UI**
- **Show real-time predictions**
- **Deploy the app locally or online**

🛠️ Tech Stack:

- **Python**
- **Streamlit**
- **Scikit-learn / TensorFlow / PyTorch**
- **Pandas / NumPy**
- **Joblib / Pickle (for model saving)**

🔄 Step-by-Step Implementation

1️⃣ Install Dependencies

```
pip install streamlit scikit-learn pandas numpy joblib
```

2️⃣ Train or Load Model

```
from sklearn.linear_model import LogisticRegression
from sklearn.datasets import load_iris
import joblib

data = load_iris()
X, y = data.data, data.target

model = LogisticRegression()
model.fit(X, y)

joblib.dump(model, "model.pkl")
```

3️⃣ Create Streamlit App

```
import streamlit as st
import numpy as np
import joblib
```

```
model = joblib.load("model.pkl")


st.title("AI Model Deployment Dashboard")

st.write("Enter input values to get predictions")
```

4️⃣ User Input Interface

```
sl = st.number_input("Sepal Length")

sw = st.number_input("Sepal Width")

pl = st.number_input("Petal Length")

pw = st.number_input("Petal Width")
```

5️⃣ Prediction Button

```
if st.button("Predict"):

   input_data = np.array([[sl, sw, pl, pw]])

   prediction = model.predict(input_data)

   st.success(f"Predicted Class: {prediction[0]}")
```

6️⃣ Run Streamlit App

```
streamlit run app.py
```

📊 Example Output:

```
Predicted Class: Setosa
```

🧠 How It Works:

- Model is trained on dataset
- Streamlit provides interactive UI
- User inputs data via dashboard
- Model returns real-time prediction

🔧 Technical Requirements:

- Train or load ML model
- Build Streamlit UI
- Accept user inputs dynamically
- Show predictions instantly
- Run app without errors

🔥 Bonus Features (Highly Recommended):

🚀 Advanced Enhancements:

- ✅ Add charts (matplotlib / seaborn)
- ✅ Display model accuracy
- ✅ Upload CSV file for batch prediction
- ✅ Add multiple ML models selector
- ✅ Deploy on Streamlit Cloud
- ✅ Add styled UI components

📦 Submission:

Include:

- Streamlit app (`app.py`)
- Trained model file
- Requirements file

- **README**

📁 Folder Structure:

Streamlit_AI_App/

├── app.py

├── model.pkl

├── requirements.txt

├── README.md

⏰ Deadline:

4–6 Hours

🎯 AI Intern – Task 23

🧠 Task Title: Create a REST API for AI Model using FastAPI / Flask

📝 Objective:

Develop a **REST API** to serve a trained **AI/ML model**, allowing external applications to send input data and receive predictions via HTTP requests.

👉 The API should handle requests and return responses in **JSON format**.

✅ Task Overview:

You will:

- Train or load an AI/ML model
- Build REST API endpoints
- Accept input data via API requests
- Return model predictions
- Test API using tools like Postman or browser

🛠️ Tech Stack:

- Python
- FastAPI **(Recommended)** or Flask
- Scikit-learn / TensorFlow / PyTorch
- Pandas / NumPy
- Uvicorn (for FastAPI)

🔄 Step-by-Step Implementation

1️⃣ Install Dependencies

pip install fastapi uvicorn scikit-learn pandas numpy joblib

2️⃣ Train & Save Model

```
from sklearn.linear_model import LogisticRegression

from sklearn.datasets import load_iris

import joblib



data = load_iris()

X, y = data.data, data.target



model = LogisticRegression()

model.fit(X, y)



joblib.dump(model, "model.pkl")
```

3️⃣ Create FastAPI App

```
from fastapi import FastAPI

import joblib

import numpy as np


app = FastAPI()


model = joblib.load("model.pkl")


@app.get("/")

def home():

  return {"message": "AI Model API is running"}
```

4 Create Prediction Endpoint

```
@app.post("/predict")

def predict(data: dict):

  features = np.array(data["features"]).reshape(1, -1)

  prediction = model.predict(features)
```

```
    return {
        "prediction": int(prediction[0])
    }
```

5️⃣ Run API Server

```
uvicorn main:app --reload
```

👉 Open in browser:
`http://127.0.0.1:8000/docs`

6️⃣ Example API Request

```
{
  "features": [5.1, 3.5, 1.4, 0.2]
}
```

📊 Example Output:

```
{
  "prediction": 0
}
```

🧠 How It Works:

- Model is trained and saved
- API loads the model

- User sends input via POST request
- Model processes input and returns prediction
- API responds in JSON format

🔧 Technical Requirements:

- Build working REST API
- Load and use trained model
- Accept input via POST request
- Return structured JSON response
- Handle errors gracefully

🔥 Bonus Features (Highly Recommended):

🚀 Advanced Enhancements:

- ✅ Input validation using Pydantic
- ✅ Multiple endpoints (predict, health check)
- ✅ Batch prediction support
- ✅ Logging system
- ✅ Dockerize API
- ✅ Deploy on cloud (Render / AWS / Railway)

📦 Submission:

Include:

- API source code
- Trained model file
- Requirements file
- README with usage instructions

📁 Folder Structure:

AI_Model_API/

├── main.py

├── model.pkl

├── requirements.txt

├── README.md

⏰ Deadline:

4–6 Hours